

MEDIATEK

MT8668 Hypervisor Camera User Manual

版本: 1.0
出版日期: 2026-01-28

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2026-01-28	张正磊	正式版

目录

版本记录.....2

目录.....3

图片目录.....4

表格目录.....4

1 Camera 虚拟化.....5

1.1 概述..... 5

1.1.1 简单介绍 5

1.1.2 名词解释 5

1.2 架构..... 6

1.2.1 Android 端 Camera 6

1.2.2 Yocto 端 Camera 7

1.3 Camera 流程 8

1.3.1 Search Sensor 流程..... 8

1.3.2 Camera Data Path – YUV Sensor..... 9

1.3.3 Camera Data Path – Raw Sensor..... 9

1.3.4 Camera Data Path – Capture..... 10

1.4 Camera Feature 简介..... 10

1.4.1 Multi-Cam 10

1.4.2 Multi-Open..... 11

1.4.3 ESD Recovery Flow 12

1.4.4 P1 直出 12

1.5 常见故障/故障分析 13

1.5.1 Camera Log CMD 13

1.5.2 Camera Dump CMD 13

1.5.3 常见 Log 分析..... 14

1.5.4 绿图/花图问题..... 15

1.5.5 查看 Camera 是否 Search 到 15

1.5.6 查看 Camera 帧率..... 15

附件一 附加条款.....16

图片目录

图 1-1. Camera 虚拟化架构6

图 1-2. Search sensor8

图 1-3. YUV camera data path9

图 1-4. Raw camera data path9

图 1-5. CaptureCamera data path10

图 1-6. ESD recovery flow12

图 1-7. P1 直出 Pipeline13

表格目录

表 1-1. 名词解释5

1 Camera 虚拟化

1.1 概述

1.1.1 简单介绍

本文档主要介绍 MT8668 Camera 虚拟化的实现原理，主要的 Camera Feature，以及常见的 Debug 分析手段。
Camera 虚拟化旨在让 Camera 可以被不同 OS（Android, Yocto, ...）无差别使用。

1.1.2 名词解释

表 1-1. 名词解释

缩略词	全称	释义
AVM	Around View Monitor	环视监控系统
Binder	—	是一种高效的进程间通信（IPC）机制，它允许数据和对象在不同的进程间传递。远程过程调用（RPC）允许一个程序调用另一个地址空间（通常是远程服务器）中的程序，就像调用本地程序一样。
DVR	Digital Video Recorder	行车记录仪
ESD	Electrostatic Discharge	Electrostatic Discharge Recovery 指的是在电子设备受到静电放电（ESD）影响后，采取的恢复措施。
ISP	Image Signal Processor	图像信号处理器，是一种用于处理数字图像信号的硬件或者软件。负责将摄像头传感器捕获的原始图像数据转成高质量的数字图像。
OS	Operating System	操作系统
VSOCK	Virtual Socket	虚拟套接字，是一种在宿主机和虚拟机（VM）之间提供双向通信的机制。它是一种专为虚拟化环境涉及的通信协议，允许宿主机和虚拟机之间以及虚拟机之间高效的数据交换。

1.2 架构

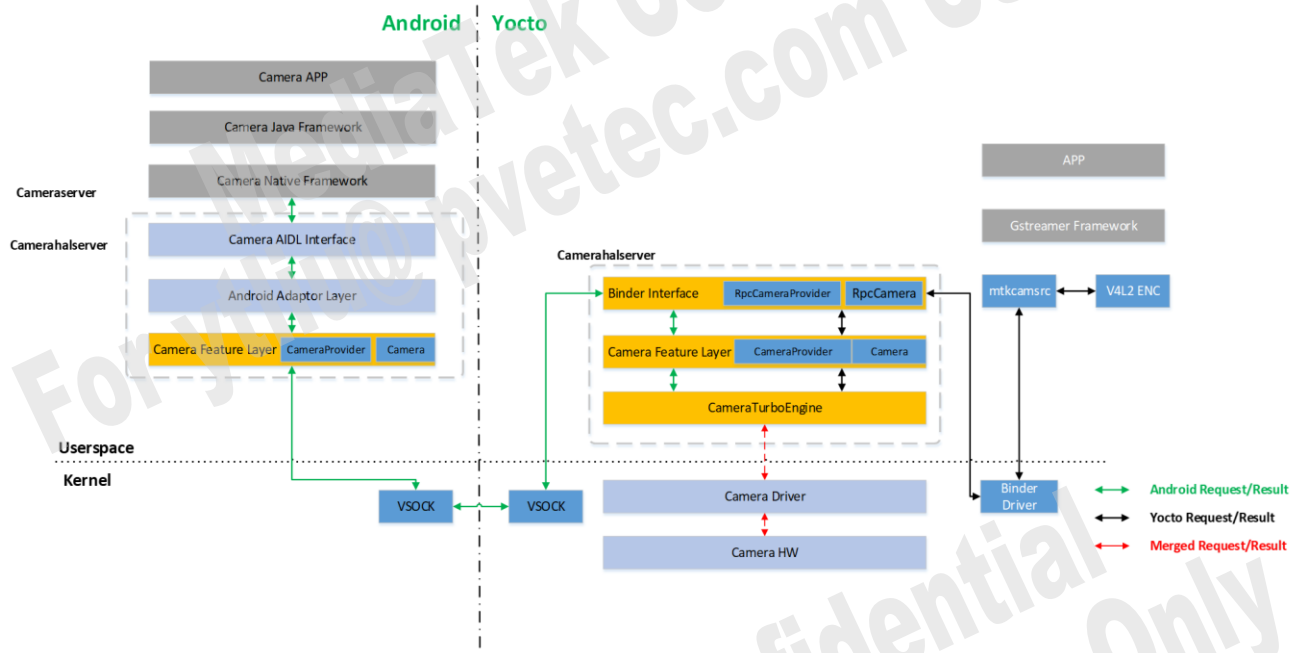


图 1-1. Camera 虚拟化架构

目前，MT8668 使用 Yocto + Android 的双系统组合，Yocto OS 作为 Host，Android OS 作为 Guest。

相应地，Yocto 端 Camera 作为 Host，所有 Camera 相关的硬件，以及对应的 driver 和 SW 逻辑控制都放在 Yocto 端；Android 端 Camera 作为 Guest，从 Yocto 端获取相关的图像数据。两者之间是基于 RpcBinder + Vsock 的方式来实现跨域通信，Android 端 Camera 跨域将 Request 发送给 Yocto 端，Yocto 端接收到 Request 后，经过处理后，将图像数据填充到 buffer 中，并将相关 Result 再跨域回传给 Android 端。

1.2.1 Android 端 Camera

Android 端 Camera 架构如图 1-1 左边所示。从上往下的层次分布依次是 APK、Java Framework、Cameraserver、Camerahalservers。除 Camerahalservers 外，其余 Layer 的实现与单 Android OS 无差别。

Camerahalservers 又大致分为 3 层：

1. AIDL Adaptor Layer
主要负责与 Cameraserver 进行 binder 交互
2. Android Adaptor Layer
主要负责处理与 Android 强相关的逻辑控制
3. Camera Feature Layer
底层 Camera 对外提供的功能接口

Android Guest 和 Android only 的 Camerahalservice 差异主要在于 Camera Feature Layer 的实现。Android only 版本, 此 layer 往下对接的是实际的 Camera 实现; 而 Android Guest 则是透过 RpcBinder + Vsock 将 Request 转嫁到 Yocto 端, 从 Yocto 获取图像数据。

1.2.2 Yocto 端 Camera

Yocto 端 Camera 架构如图 1-1 右边所示。

Yocto Camera 作为 Host, 在 Yocto 启动时自启动一个常驻进程 Camerahalservice。

从上到下大致分为 5 层:

1. Binder Interface Layer

用于提供 binder 服务

- a) Camerahalservice 在启动时会提供两个 service (LocalService/RpcService)
- b) LocalService 采用 IPC binder 的方式, 为 Yocto 本地进程提供服务, 如 Gst、FastRvc
- c) RpcService 采用 RpcBinder + vsock 的方式, 为 Android 远端进程提供服务, 如 android camerahalservice

2. Camera Feature Layer

对外提供功能接口

- d) CameraProvider 作为一个 static 对象, 用于管控所有 Camera, 对外提供静态信息, 如 cameraIdList、static metadata 等
- e) 当用户打开 Camera 时, 就会生成一个 Camera 对象, 提供相应的控制接口, 如 configure()、pushRequest()、flush()、close()等

3. CameraTurboEngine

用于 Camera 的 SW 逻辑控制, 并提供 Low MIPS、Multi-Cam、Multi-Open 等强大功能

4. Camera Driver

用于控制 Sensor、ISP 等硬件

5. Camera HW

主要包括 Sensor、ISP 等硬件

无论是远端进程, 还是本地进程, 都是透过 Binder Interface Layer 使用 CameraProvider/Camera 来获取图像数据。

Camerahalservice 的处理流程如下:

- 1. 通过 CameraProvider::listAvailableCameras()获取可用的 Camera ID List
- 2. 通过 CameraProvider::open()接口打开指定的 Camera, 并生成一个 Camera 对象
- 3. 通过 Camera::setCallback()设置回传

4. Camera::configures()会根据用户带进来的 Streams，配置好相关的 Pipeline
5. Camera::pushRequest() 将用户带进来的 Request (buffer & settings) 丢进 Pipeline 做一系列处理，包括 P1 从 Sensor 收图，然后经 P2 后处理后，填入用户 buffer 中
6. 使用回调接口将 Result 回传给用户
7. 使用完成后，通过 Cameraprovider::flush()&close()接口关闭 Camera

1.3 Camera 流程

1.3.1 Search Sensor 流程

Yocto 端 Camerahalserver 启动后会进行 Search Sensor 的工作，search sensor flow 如图 1-2 所示。Yocto search 到 sensor 后，会注册 Rpc Service。Android Camerahalserver 启动后，获取 Yocto 端 Rpc Service。

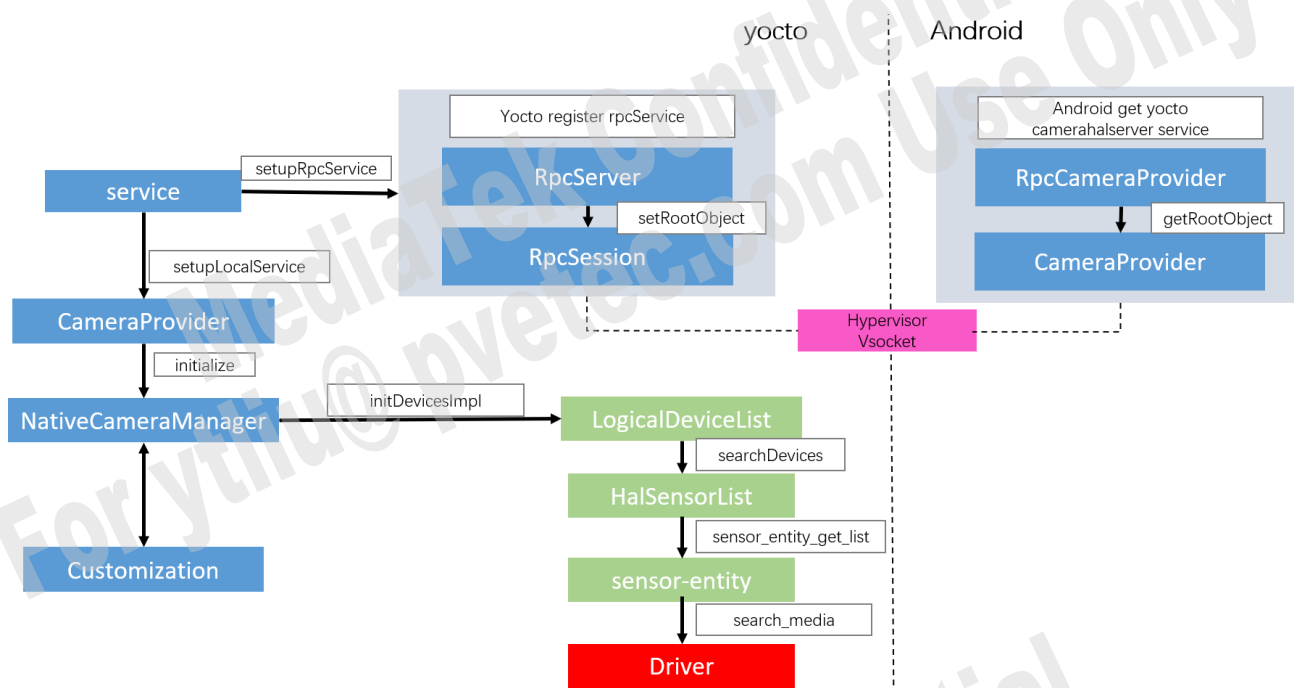


图 1-2. Search sensor

1.3.2 Camera Data Path – YUV Sensor

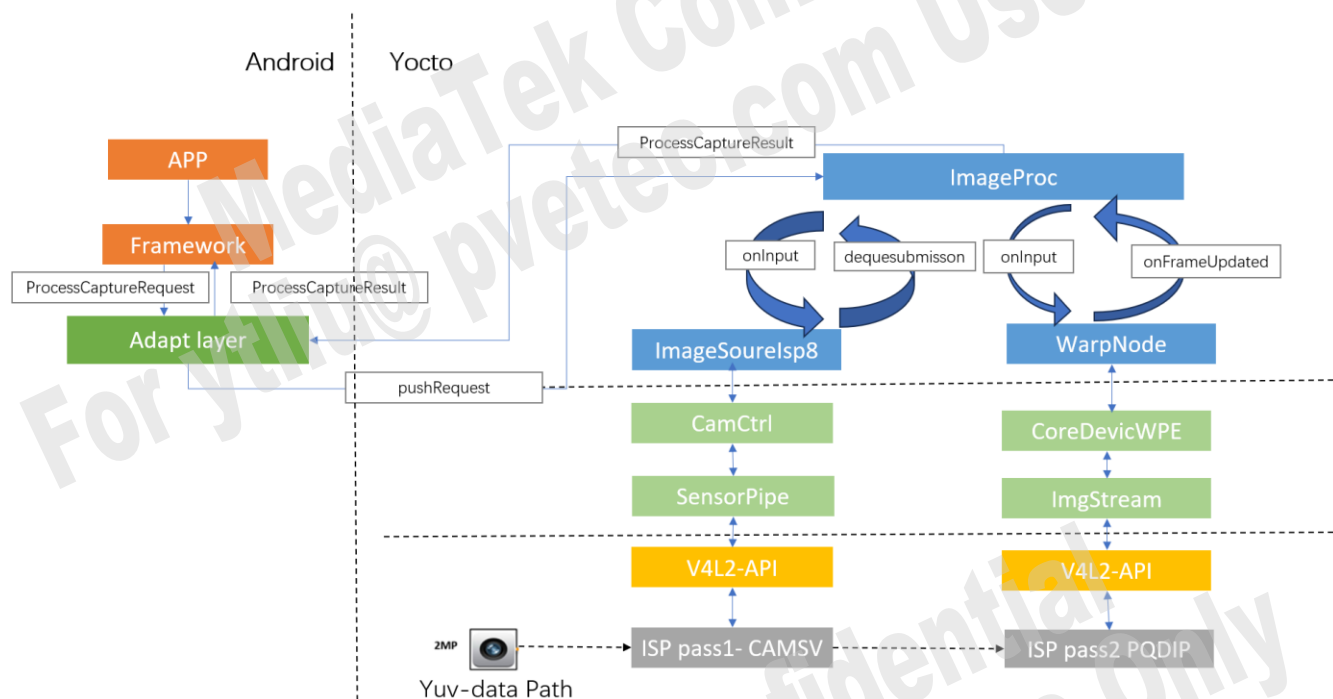


图 1-3. YUV camera data path

1.3.3 Camera Data Path – Raw Sensor

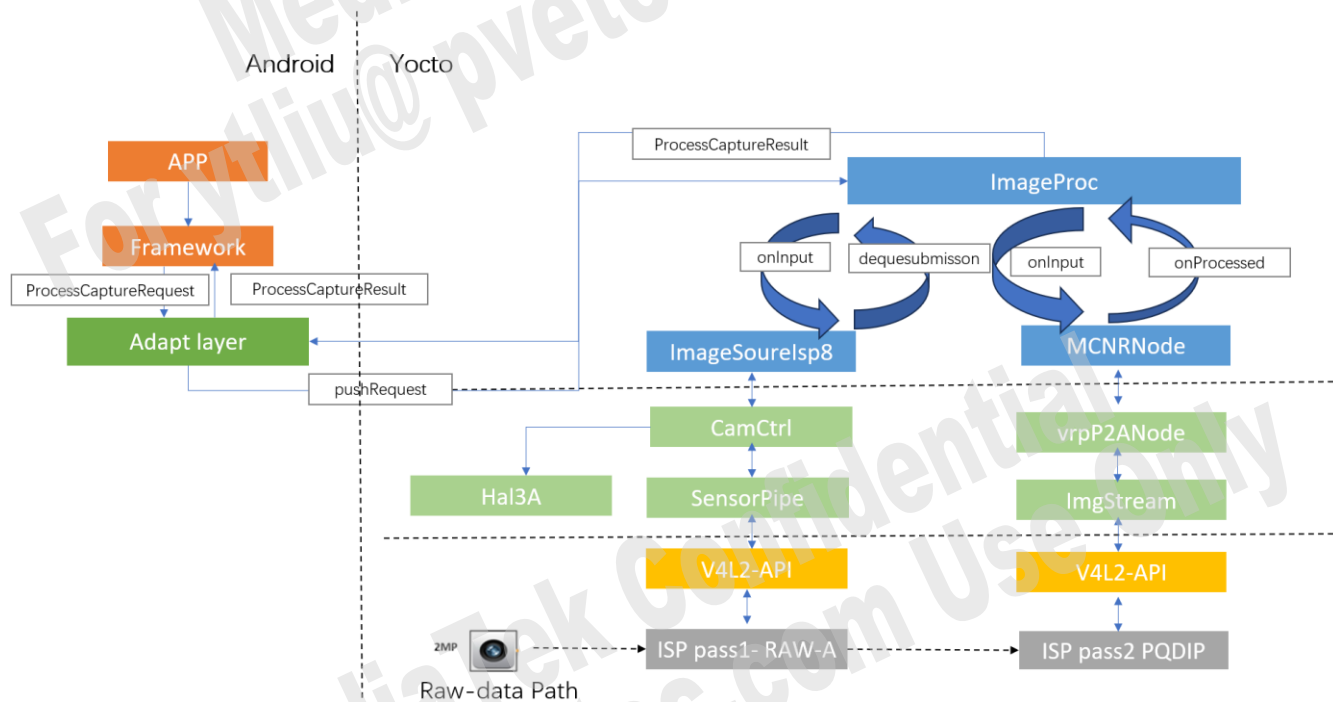


图 1-4. Raw camera data path

1.3.4 Camera Data Path – Capture

如图 1-5 所示，Camera Data Path 在之前的基础上，增加了 JpegNode → JpegCodec → JpegEncode 相关 flow；其中 jpeg encode 使用 HW encode。

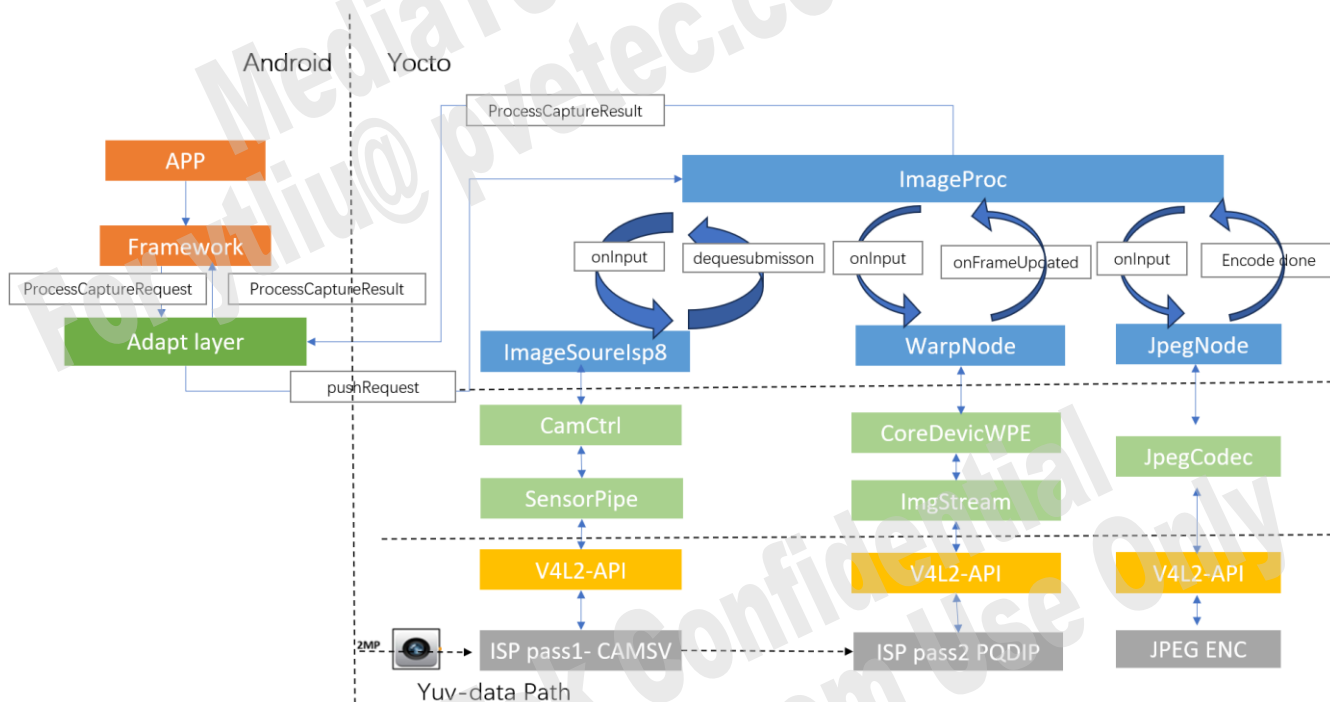


图 1-5. CaptureCamera data path

1.4 Camera Feature 简介

本章简要地介绍 MT8668 提供的一些车机特有的 Camera Feature。

1.4.1 Multi-Cam

目前 MT8668 公版配置了 1×AVM + 4×MultiYuv 的 Camera 组合，其中 4×MultiYuv 是通过一个 SerDes 外接的 4 颗摄像头，每颗摄像头可被不同 APK 单独控制。

每个 Camera 有其特定的用途，如 AVM、DVR 等，不同应用需要打开指定的 Camera，所以需要获知每颗 Camera 的用途。为此，我们增加了一个新的 Metadata VendorTag (MTK_SENSOR_FEATURE_SENSOR_TYPE) 来标识 Camera 的身份；配置 sensor 的 Static Metadata 时设置 sensor type 值，应用再通过 Static Metadata 查询该 VendorTag 即可获知 Camera 的用途。

```

/**
 * This tag is used to inform the app about the sensor type
 *
 * @par Type
 *      int32_t
 * @par Category
 *      Static
 * @par Valid Timing
 *      Launch
 * @version Added since Metadata Tag V1.3.
 */
MTK_SENSOR_FEATURE_SENSOR_TYPE,

```

```

typedef enum mtk_camera_metadata_enum_sensor_feature_sensor_type {
    MTK_SENSOR_FEATURE_SENSOR_TYPE_NORMAL,
    MTK_SENSOR_FEATURE_SENSOR_TYPE_AVM,
    MTK_SENSOR_FEATURE_SENSOR_TYPE_DMS,
    MTK_SENSOR_FEATURE_SENSOR_TYPE_OMS,
    MTK_SENSOR_FEATURE_SENSOR_TYPE_IMS,
    MTK_SENSOR_FEATURE_SENSOR_TYPE_DVR,
    MTK_SENSOR_FEATURE_SENSOR_TYPE_MULTI_EXT0,
    MTK_SENSOR_FEATURE_SENSOR_TYPE_MULTI_EXT1,
    MTK_SENSOR_FEATURE_SENSOR_TYPE_MULTI_EXT2,
    MTK_SENSOR_FEATURE_SENSOR_TYPE_MULTI_EXT3,
} mtk_camera_metadata_enum_sensor_feature_sensor_type_t;

```

1.4.2 Multi-Open

MT8668 支持同颗 Camera 被多个应用同时打开，可以是同 OS 的应用，也可以是不同 OS 的应用。

当多个 User 打开同一个 Camera 时，操作流程如图 1-1 所示。

1. 每个 User 在 Open 时会获取到独有的 Camera 实例
2. Config 阶段，CameraTurboEngine 会根据多 User 提供的 config 信息构建兼容的底层 Pipeline
3. Request 阶段，CameraTurboEngine 会将多 User 的 request 做 merge 操作，然后丢进 Pipeline 做处理
4. Pipeline 处理完成后，CameraTurboEngine 又会将 Result 分别派给不同的 User

应以下各种不同的需求，我们增加了 Camera ID 定制化的逻辑，用户可以根据自己的需求修改

CustLayer::listAvailableCameras()接口。文件目录在 yocto: src/multimedia/mtkcam-mt8668/custLayer/下。

- Android、Yocto 应用希望看到不同的 cameraIdList
- 在不修改 AOSP flow 的情况下，Android 的多个 APK 对同颗 Camera 有多开需求

```

bool CustLayer::listAvailableCameras(std::vector<int32_t>& list, int32_t strategy) {
    std::unique_lock<std::mutex> _l(mLock);
    TINY_LOGD("listAvailableCameras, strategy %d", strategy);
    #if CAMERA_ID_CUSTOMIZATION_ENABLE
    // sample code:
    // if want to customize camera id to android & yocto
    // can ref code like this:
    std::vector<int32_t> result = {};
    if (strategy == ANDROID) {
        for (auto& id : list) {
            result.emplace_back(id);
            auto type = getCameraType(id);
            auto it = sMultiOpenList.find(type);
            if (it != sMultiOpenList.end()) {
                auto cnt = it->second;
                if (cnt < 0 || cnt > 3) {
                    TINY_LOGE("invalid multiopen count %d for Cam: %d", cnt, id);
                    continue;
                }
                for (auto i = 0; i < cnt - 1; i++) {
                    auto virtualId = ((1 + i) << 4) + id;
                    result.emplace_back(virtualId);
                    TINY_LOGI("generate virtual camid %d for Cam: %d", virtualId, id);
                }
            }
        }
        list.clear();
        list.assign(result.begin(), result.end());
    } else if (strategy == YOCTO) {
        for (auto& camid : list) {
            camid += CAMERA_ID_SHIFT;
        }
    } else {
        TINY_LOGD("others, do nothing");
    }
    #endif
    return true;
}

```

1.4.3 ESD Recovery Flow

针对 Sensor MIPI 信号可能收到受到静电干扰导致 MIPI 信号异常，ISP 收不到数据的问题，我们做了 ESD Recovery 机制。

ESD Recovery 的逻辑控制如下：

1. ISP driver 启用 thread 实时监控 Sensor SOF
2. 如果有 SOF，走正常流程，通知 SOF_EVENT
3. 如果没有 SOF，走异常流程，通知 V4L2_EVENT_ESD_RECOVERY 到 MW
4. MW 收到 V4L2_EVENT_ESD_RECOVERY 事件后，检查是否需要 reset sensor，默认 reset 3 次
5. MW 调用 V4L2_CID_MTK_SENSOR_RESET 对 Sensor 进行 reset
6. 如果 reset sensor 后，ISP 可以检测到 SOF，则表示 Sensor 恢复，继续走 OK flow
7. 如果 reset 3 次后，ISP 依旧检测不到 SOF，则说明 Sensor 无法恢复，MW 通知 DEVICE_ERROR 到上层
8. 上层收到 DEVICE_ERROR 后，走 close camera flow

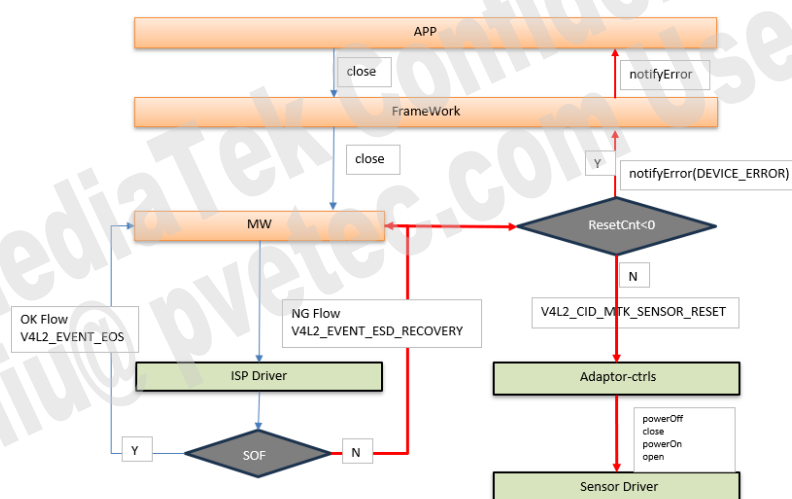


图 1-6. ESD recovery flow

1.4.4 P1 直出

针对 YUV Sensor，应用下发的 Stream 在满足以下特定的条件下，APP 所要求的数据即可直接从 P1 出，bypass P2，达到 Low MIPS 和 Low Latency 的需求。

- 只有一路 Stream
- Stream format 为 YUYV，size 跟 sensor size 一致

注：目前 MT8668 YUV Sensor 的 format 不论是哪种 order，从 P1 输出时，会被 P1 driver 固定 swap 成 YUYV 的 order，所以，APP 需要 P1 直出是要下发 YUYV 的 format。



图 1-7. P1 直出 Pipeline

1.5 常见故障/故障分析

1.5.1 Camera Log CMD

1. Android Log
N/A

2. Yocto Log
总开关:

```
adb -s YOCTO shell setprop persist.mtk.camera.log_level N
```

N 可设置成 0~4 中的一个，表示输出何种级别的 log。当前默认值为 3。

- 0/1: LOGE/LOGW 级别
- 2: LOGI 级别
- 3: LOGD 级别
- 4: LOGV 级别

注意，设置该 log level，需要重启 camerahalerver 后才能生效。

RpcCamera:

```
adb shell "mtkcam_setprop vendor.debug.camera.rpclog 1"
```

ImageNode:

```
adb shell mtkcam_setprop vendor.debug.camera.rpclog 1
```

CamCtrl:

```
adb shell mtkcam_setprop vendor.debug.camera.camctrl.log_level 2
```

Log 默认存放在 /data/debuglogger/mobilelog/下。

1.5.2 Camera Dump CMD

Raw Sensor 和 YUV Sensor 的 data path 不一样，相应的 dump 位置和方式也不同；

1) Raw Sensor

```
// open camera 前
mkdir -p /data/vendor/camera_dump
adb shell "mtkcam_setprop vendor.debug.camera.vrp.dump 2"
```

```
// 运行时 (1:start dump; 0: stop dump)
adb shell "mtkcam_setprop vendor.debug.camera.vrp.dump.start N"
```

2) YUV Sensor

```
// open camera 前
mkdir -p /data/vendor/camera_dump
chmod 777 /data/vendor/camera_dump
adb shell "mtkcam_setprop vendor.debug.camera.coredevice.wpe.dump 2"
```

1.5.3 常见 Log 分析

Camera 各阶段 (open -> configure -> request -> flush -> close) 的关键 log 如下:

```
// open
2024-10-14 04:18:41.735688 2520 D camerahalserver: 0x7faa172-0x [3573][RpcCameraProvider] [open] [-1--1] open Camera 0x7efc011990, cam::0, userId 0
// configure +
2024-10-14 04:18:41.814524 2520 I camerahalserver: 0x7faa172-0x [3573][RpcCamera] [configure] Cam::0, userId 0 +
// 当前 Pipeline 的 Topology
2024-10-14 04:18:41.815614 2520 D camerahalserver: 0x7faa172-0x [3573][TINY_PIPELINE] Head Node : P1Node[0] phyId 0
2024-10-14 04:18:41.815672 2520 D camerahalserver: 0x7faa172-0x [3573][TINY_PIPELINE] Node Topology : P1Node[0] -> WarpPQNode[2]
2024-10-14 04:18:41.815765 2520 D camerahalserver: 0x7faa172-0x [3573][TINY_PIPELINE] Node Topology : WarpPQNode[2] -> JpegNode[1]
2024-10-14 04:18:41.815825 2520 D camerahalserver: 0x7faa172-0x [3573][TINY_PIPELINE] Node Topology : P1Node[0] -> WarpPQNode[3]
// 每个 Node 的 in/out stream
2024-10-14 04:18:41.844046 2520 D camerahalserver: 0x7faa172-0x [3573][TINYMW/ConfigStreamInfoPolicy] [operator()] [P1-SBP] P1Node(0) out
streamId:f00ffffff
2024-10-14 04:18:41.844603 2520 D camerahalserver: 0x7faa172-0x [3573][TINYMW/ConfigStreamInfoPolicy] [configWarpPQ] [VSS] [NonP1-SBP] WarpPq(2) in
streamId:f00ffffff
2024-10-14 04:18:41.844633 2520 D camerahalserver: 0x7faa172-0x [3573][TINYMW/ConfigStreamInfoPolicy] [operator()] [NonP1-SBP] WarpPq(2) out
streamId:2001800ffffff
2024-10-14 04:18:41.844649 2520 D camerahalserver: 0x7faa172-0x [3573][TINYMW/ConfigStreamInfoPolicy] [operator()] [NonP1-SBP] WarpPq(2) out
streamId:2001900ffffff
2024-10-14 04:18:41.844895 2520 D camerahalserver: 0x7faa172-0x [3573][TINYMW/ConfigStreamInfoPolicy] [configWarpPQ] [NonP1-SBP] WarpPq(3) in
streamId:f00ffffff
2024-10-14 04:18:41.844910 2520 D camerahalserver: 0x7faa172-0x [3573][TINYMW/ConfigStreamInfoPolicy] [operator()] [NonP1-SBP] WarpPq(3) out
streamId:3001a0000000
2024-10-14 04:18:41.844926 2520 D camerahalserver: 0x7faa172-0x [3573][TINYMW/ConfigStreamInfoPolicy] [operator()] [NonP1-SBP] WarpPq(3) out
streamId:3001b0000000
2024-10-14 04:18:41.844724 2520 D camerahalserver: 0x7faa172-0x [3573][TINYMW/ConfigStreamInfoPolicy] [operator()] [NonP1-SBP] Jpeg(1) out
streamId:100ff0000000
2024-10-14 04:18:41.844756 2520 D camerahalserver: 0x7faa172-0x [3573][TINYMW/ConfigStreamInfoPolicy] [configJPEG] [NonP1-SBP] Jpeg(1) in
streamId:2001800ffffff
2024-10-14 04:18:41.844771 2520 D camerahalserver: 0x7faa172-0x [3573][TINYMW/ConfigStreamInfoPolicy] [configJPEG] [NonP1-SBP] Jpeg(1) in
streamId:2001900ffffff
// User 配置的各路 streaminfo
2024-10-14 04:18:41.893378 2520 D camerahalserver: 0x7faa172-0x [3573][RpcCamera] [collectStreamInfo] sid 0, (size: 1280x720)-(usage: 131379)-(fmt:
17 -> 17)-(dataSpace: 0x8c20000), Plane: [ 1280/921600 1280/460800 ]
2024-10-14 04:18:41.893390 2520 D camerahalserver: 0x7faa172-0x [3573][RpcCamera] [collectStreamInfo] sid 1, (size: 2928640x1)-(usage: 131123)-
(fmt: 33 -> 8960)-(dataSpace: 0x8c20000), Plane: [ 4720640/4720640 ]
2024-10-14 04:18:41.893403 2520 D camerahalserver: 0x7faa172-0x [3573][RpcCamera] [collectStreamInfo] sid 2, (size: 176x144)-(usage: 131123)-(fmt:
17 -> 17)-(dataSpace: 0x8c20000), Plane: [ 176/25344 176/12672 ]
// configure -
2024-10-14 04:18:41.893439 2520 I camerahalserver: 0x7faa172-0x [3573][RpcCamera] [configure] Cam::0, userId 0 -
// pushRequest
2024-10-14 04:18:41.914048 2520 D camerahalserver: 0x7faa172-0x [3573][TINY_NATIVECAMERA] [pushRequest] Cam::0, User 0, [NODEDBG_FRAME R:0,F:-1]
2024-10-14 04:18:41.914123 2520 D camerahalserver: 0x7faa172-0x [3573][TINY_NATIVECAMERA] [pushRequest] Cam::0, User 0, pushReq:0 -
2024-10-14 04:18:41.919035 2520 I camerahalserver: 0x7faa172-0x [3910][ImageSource] [ImgSrc_256] ( 613) [onInput] [Cam::0] initReqCnt(2/ 4 / 4)
// P1 process done
2024-10-14 04:18:42.011746 2520 D camerahalserver: 0x7faa172-0x [3919][TINY_IMAGENODE] [P1Node_0] ( 285) [requestDispatch] [Cam::0] [NODEDBG_FRAME
R:0,F:0] out, result(0), #processing(3)
// p2 process done
2024-10-14 04:18:42.029380 2520 D camerahalserver: 0x7faa172-0x [3927][TINY_IMAGENODE] [WarpPQNode_3] ( 285) [requestDispatch] [Cam::0] [NODEDBG_FRAME
R:0,F:0] out, result(0), #processing(0)
// flush
2024-10-14 04:18:45.485391 2520 I camerahalserver: 0x7faa172-0x [3573][RpcCamera] [flush] Cam::0, userId 0, flush
// close
2024-10-14 04:18:45.540466 2520 I camerahalserver: 0x7faa172-0x [3573][RpcCamera] [close] Cam::0, userId 0, close +
2024-10-14 04:18:45.542418 2520 I camerahalserver: 0x7faa172-0x [3573][RpcCamera] [close] Cam::0, userId 0, close -
2024-10-14 04:18:45.542981 2520 D camerahalserver: 0x7faa172-0x [3573][RpcCameraProvider] [close] close Camera 0x7efc011990, cam::0, userId 0
```

1.5.4 绿图/花图问题

若出现绿图花图问题，可先按照 1.5.2 节介绍的 dump 方式，理清哪个阶段出的问题。

- 1) input 有问题，先从 P1 driver 和 Sensor 端理清。
- 2) input 正常，output 有问题，要从 P2 driver 分析。
- 3) input/output 都正常，那说明跟 Camera 无关，需要从 Camera 后一级 Module 理清。

1.5.5 查看 Camera 是否 Search 到

若出现 Camera 应用无法打开，首先检查是否有 search sensor 成功，查看 log：如下 log 显示有接 5 个 Camera；其中：SENSOR_DRVNAME_MAX96712_MIPI_YUV 是 AVM Camera，SENSOR_DRVNAME_MAX96712A0_MIPI_YUV 属于 Multicam 中的一个。

```
2024-02-28 01:26:16.980829    2520 D camerahalserver: [2736] [TINY_NATIVECAMERAMANAGER] [0]
vid: 0 isHidden:0 sensor:SENSOR_DRVNAME_MAX96712_MIPI_YUV
2024-02-28 01:26:16.994514    2520 D camerahalserver: [2736] [TINY_NATIVECAMERAMANAGER] [1]
vid: 1 isHidden:0 sensor:SENSOR_DRVNAME_MAX96712A0_MIPI_YUV
2024-02-28 01:26:17.011536    2520 D camerahalserver: [2736] [TINY_NATIVECAMERAMANAGER] [2]
vid: 2 isHidden:0 sensor:SENSOR_DRVNAME_MAX96712A1_MIPI_YUV
2024-02-28 01:26:17.028270    2520 D camerahalserver: [2736] [TINY_NATIVECAMERAMANAGER] [3]
vid: 3 isHidden:0 sensor:SENSOR_DRVNAME_MAX96712A2_MIPI_YUV
2024-02-28 01:26:17.045203    2520 D camerahalserver: [2736] [TINY_NATIVECAMERAMANAGER] [4]
vid: 4 isHidden:0 sensor:SENSOR_DRVNAME_MAX96712A3_MIPI_YUV
2024-02-28 01:26:17.059735    2520 D camerahalserver: [2736] [TINY_NATIVECAMERAMANAGER] [5]
vid: 5 isHidden:1 sensor:SENSOR_DRVNAME_MAX96712A_MIPI_YUV
```

1.5.6 查看 Camera 帧率

查看 Camera 帧率是否正常，需要打开 log：

```
adb shell mtkcam_setprop vendor.debug.tinycam.isp 32
2024-10-01 06:20:37.754781    2523 I camerahalserver: [4411] [TINY_IMAGENODE]
[P1Node_0] ( 643) [estimateFps] [Cam::0] fps : 28.92 / sec
2024-10-01 06:20:37.762614    2523 I camerahalserver: [4419] [TINY_IMAGENODE]
[WarpPQNode_3] ( 643) [estimateFps] [Cam::0] fps : 29.05 / sec
2024-10-01 06:20:38.783951    2523 I camerahalserver: [4411] [TINY_IMAGENODE]
[P1Node_0] ( 643) [estimateFps] [Cam::0] fps : 29.99 / sec
2024-10-01 06:20:38.795557    2523 I camerahalserver: [4419] [TINY_IMAGENODE]
[WarpPQNode_3] ( 643) [estimateFps] [Cam::0] fps : 29.86 / sec
2024-10-01 06:20:39.817174    2523 I camerahalserver: [4411] [TINY_IMAGENODE]
[P1Node_0] ( 643) [estimateFps] [Cam::0] fps : 29.99 / sec
2024-10-01 06:20:39.825135    2523 I camerahalserver: [4419] [TINY_IMAGENODE]
[WarpPQNode_3] ( 643) [estimateFps] [Cam::0] fps : 29.98 / sec
```

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿赔偿责任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。